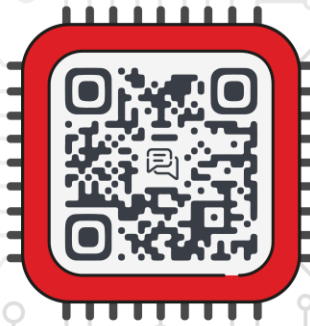




9



4



480 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

collections

د. عملي مشترك



RB Informatics; 11/12/2025

برمجة ٣

ال collections هي مجموعة واجهات و كلاسات جاهزة بجافا لتخزين البيانات فيها و نديرها بكفاءة .

من ال collections :

Map _ Queue _ Set _ List

قد تتساءل لماذا إذن لا نستخدم مصفوفة عادية بدلا من ال collection فالمصفوفة أيضا تخزن البيانات ؟
وذلك لأن المصفوفات ثابتة كما نعلم أي لن نتمكن من إضافة عناصر أو تغيير حجم هذه المصفوفة لاحقا و أيضا لا تتوفر عمليات البحث و الإضافة و الترتيب الجاهزة كال collections

١. List

بداية ال List تسمح بالتكرار و هي مرتبة و يمكننا الوصول للعنصر الذي داخلها من خلال رقمه .

١. ArrayList

```
ArrayList <String> names = new ArrayList<>();
names.add("Adam");
names.add("Sara");
names.add("Sara");
```

إذا هنا أنشئنا ArrayList وهي تضم فقط الأسماء ومن ثم أضفنا اسم داخل هذه ال ArrayList ولاحظ أننا أضفنا نفس الاسم للمجموعة مرتين و هذا مسموح هنا أي أننا لو طبعنا الأسماء سيتم طباعة ثلاث أسماء أي اسم مكرر.

```
System.out.println(names.get(0));
```

إذا للوصول للعناصر:

هنا سيتم طباعة العنصر الأول من ال arrayList .

```
names.remove(1);
```

أما للحذف فنكتب:

فيحذف العنصر ذو ال index الأول .



٢. LinkedList :

```
LinkedList<Integer> nums = new LinkedList<>();
nums.add(10);
nums.add(20);
```

الفرق بين ال ArrayList وال LinkedList

LinkedList	ArrayList
بطيئة بالوصول لعنصر	وصول سريع للعنصر
عقد مرتبطة ببعضها البعض	هي عبارة عن مصفوفة
حذف / إضافة سريعة للعنصر	بطيئة بحذف العناصر

٢. مجموعات ال Set:

وهي مجموعات لا تسمح بالتكرار ولا تعتمد على ال index .

١. HashSet

```
HashSet<String> set = new HashSet<>();
set.add("Java");
set.add("C++");
set.add("Java");
```

لو اردنا طباعة الخرج سيكون :

Java

C++

إذا التكرار يحذف تلقائيا ومن الممكن أن يكون الخرج غير مرتب .

٢. TreeSet

```
TreeSet<Integer> t = new TreeSet<>();
t.add(30);
t.add(10);
t.add(20);
```

الخرج لو أردنا طباعته : 10 20 30

إذا تترتب ال TreeSet تلقائياً ولكنها أبطأ من ال HashSet .

٣. Queue

FIFO

المبدأ الخاص بها

First In First Out



مثال :

```
Queue<String> q = new LinkedList<>();
q.add("A");
q.add("B");
q.add("C");
```

- من الدوال المهمة للطواير دوال حذف أول عنصر و عرض أول عنصر :
- q.poll() لحذف أول عنصر
- q.peek() لعرض أول عنصر بدون حذفه

٤. Map :

تخزن عناصرها على شكل Key, Value , ولا يمكننا تكرار المفاتيح .

1. HashMap :

```
HashMap<Integer,String> map = new
HashMap<>();
map.put(1,"Java");
map.put(2,"C++");
map.put(1,"Python");
```

لاحظ، أننا كررنا المفتاح 1 ولكن هذا غير مسموح إذا ما سيحدث أنه ستلغى القيمة القديمة و تستبدل بالجديدة

إذا النتيجة ستكون:

ملاحظة : ال HashMap لا تتضمن الترتيب قد تظهر النتيجة في الطباعة
بترتيب مختلف (مثلاً 2 قبل 1) إلا إذا استخدمنا LinkedHashMap

1 Python

2 C++

هنا 2 هو المفتاح

```
System.out.println(map.get(2));
```

وللوصول للقيم نستخدم :

■ ملاحظة: يمكننا ترك الاقواس <> فارغة عندئذ يمكن لل HashMap ان تستقبل أي نوع من المتغيرات .

مثال مهم:

لدينا النص التالي : String text = "abcdabcdaabcdf"; نريد معرفة عدد مرات تكرار كل حرف character ؟

الحل:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        String text = "abcdabcdaabcdf";
        HashMap<Character, Integer> map = new HashMap<>();
        // ملاحظة : نكتب char وليس Character ونفس الشيء لبقية الانواع
        // نحتاج حلقة تكرارية لتمر على جميع محرف النص و نخزنهم في c
        for (Character c : text.toCharArray()) {
            // في key=c نعيد لنا قيمة العنصر ذو ال map.getOrDefault(c,0) و text تمثل المحرف من النص c
            // في حال وجوده في ال HashMap او يعيد لنا قيمة افتراضية 0 Default وهي الصفر
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        System.out.println(map);
    }
}
```

OutPut : a=4 , b=3 , c=3 , d=3 , f=1 .

ملاحظة: يوجد توابع لمعرفة اكبر قيمة بال map وأصغر قيمة وتستخدم كالآتي :

الأكبر : Collections.max(map.values());

الأصغر : Collections.min(map.values());



System.out.println("THE END- for real this time");

GOOD LUCK